

VisionEngine

VisionEngine

Tagline: See the UI like a user — computer vision plus LLM vision for analysis and navigation.

Summary: VisionEngine is a decoupled Go toolkit that combines classic computer vision with LLM-based vision to analyze user interfaces, detect UI elements and visual issues, and build navigation graphs of app screen transitions — with pluggable multi-provider vision backends and OpenCV gated behind a build tag.

Short description (~40 words): A reusable Go module for UI analysis and navigation-graph construction. It offers an analyzer layer (UI elements, screen diffs, visual issues), a navigation graph with BFS pathfinding and DOT/JSON/Mermaid export, and LLM-vision adapters for GPT-4o, Claude, Gemini, Qwen-VL, and more.

Long description (150-250 words): VisionEngine gives automation the ability to perceive and reason about user interfaces the way a person would. It is organized into four cooperating layers. The **Analyzer** defines interfaces (Analyzer, VideoProcessor) and value types (UIElement, ScreenAnalysis, ScreenDiff, Rect, Size, TextRegion, VisualIssue, ScreenIdentity, Action, KeyFrame) with a StubAnalyzer reference implementation, so consumers can analyze screens and diffs against a stable contract. The **NavigationGraph** models an app as a directed graph of screen transitions with BFS pathfinding and three export backends (DOT, JSON, Mermaid), and it ships stress/automation/integration/security test suites. The **LLM Vision** layer defines a VisionProvider interface plus adapters for OpenAI (GPT-4o), Anthropic (Claude), Gemini, Qwen-VL, Kimi, StepGUI, Astica, and Ollama, composed via a FallbackChain so a failing or weak provider degrades gracefully to the next. A **Configuration** layer handles env-var loading and validation with all user-facing error strings routed through the `i18n.Translator` seam. Crucially, heavy native dependencies are optional: OpenCV bindings are build-tag-gated (-tags vision), and the default build ships stubs so the entire module is buildable and testable on any Go 1.25+ host without OpenCV. VisionEngine is fully decoupled per the constitution

(CONST-051(B)) and is incorporated by consumers — notably HelixQA — as an equal-codebase submodule for visual, evidence-driven UI testing.

Why we built it: UI test automation that relies only on accessibility trees or selectors misses what the user actually sees. VisionEngine adds real visual understanding — element detection, screen diffing, and LLM-vision reasoning — plus a navigable map of app screens, so automation can both perceive and route through a UI.

Why it's a game-changer: It fuses deterministic computer vision with modern multimodal LLM vision behind one interface and a fallback chain, and makes it adoptable anywhere by keeping OpenCV optional — so any Go project gains UI perception without heavy native prerequisites.

What's innovative: - Dual perception: classic CV (OpenCV/GoCV) plus multi-provider LLM vision with a fallback chain. - Navigation graph with BFS pathfinding and DOT/JSON/Mermaid export. - Build-tag-gated OpenCV so the module stays buildable/testable without native deps. - Fully decoupled, i18n-seamed, equal-codebase submodule (used by HelixQA).

Biggest technical challenges + how solved: - *Heavy native dependency friction:* solved with `-tags vision` gating and default stubs so CI/hosts without OpenCV still build and test. - *Vision-provider unreliability:* solved with a `VisionProvider` interface and `FallbackChain` composer. - *Mapping complex app flows:* solved with a directed navigation graph plus BFS pathfinding and multi-format export. - *Coupling:* solved via CONST-051(B) decoupling and the i18n translator seam.

Tech stack (why + how): - **Go (1.25+)** — module core and all four layers. - **GoCV / OpenCV** — classic computer vision, build-tag-gated. - **LLM vision providers (GPT-4o, Claude, Gemini, Qwen-VL, Kimi, StepGUI, Astica, Ollama)** — multimodal UI reasoning via adapters. - **Graph algorithms (BFS)** — navigation pathfinding. - **DOT / JSON / Mermaid exporters** — navigation-graph visualization. - **i18n Translator** — decoupled user-facing strings.

Public links: - GitHub (HelixDevelopment): <https://github.com/HelixDevelopment/VisionEngine> (public). - GitHub (vasic-digital): <https://github.com/vasic-digital/VisionEngine> (public). - Consumed by HelixQA as an equal-codebase submodule.

Suggested diagrams/illustrations (OpenDesign): 1. Four-layer stack: Analyzer / NavigationGraph / LLM Vision / Config. 2. Navigation graph rendered (Mermaid-style) with BFS path highlighted. 3. Vision fallback chain across providers. 4. Build-tag split: default stub build vs `-tags vision` OpenCV build.

Site relevance: vasic.digital (multimodal AI / QA-automation depth). Complements the HelixQA anti-bluff story.

Priority tier: vasic-util-secondary

Source provenance: `gh repo view vasic-digital/VisionEngine`
README (four layers, value types, providers, FallbackChain,
build-tag gating, CONST-051); `_analysis/github-helix-others.md`
(VisionEngine listing); `_analysis/github-vasic-digital.md` (size/
description).